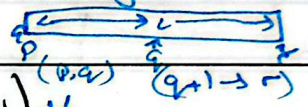
ıy/date

Merge Sort

It follows the divide-and-conquer paradigm.

Algorithm:

```
1  MergeSort (A, p, r)
2
3  IF p<r THEN
4      q = ⌊(p+r)/2⌋ # integer division.
5      MergeSort (A, p, q) #
6      MergeSort (A, q+1, r) #
7      Merge (A, p, q, r)
8
9
10 Merge (A, p, q, r)
11
12     n1 = q - p + 1
13     n2 = r - q
14
15     let L[1...n1+1] and R[1...n2+1] be new
16     arrays.
17
18
```



BHATTI

CamScanner

day/date

```
FOR  i = 1   TO  n1
     L[i] = A[p+i-1]

FOR  j = 1   TO  n2
     R[j] = A[q+j]

L[n1+1] = ∞
R[n2+1] = ∞

i = 1
j = 1

FOR k = p  TO  r

     IF L[i] ≤ R[j]  THEN
          A[k] = L[i]
          i = i+1

     ELSE
          A[k] = R[j]
          j = j+1
```

BHATTI

CamScanner

/date

Understanding Algo through dry run:

e.g:

A = [10, 4, 6, 3, 8, 2, 5, 7]

mergeSort (A, 1, 8)

p = start index

q = end index.

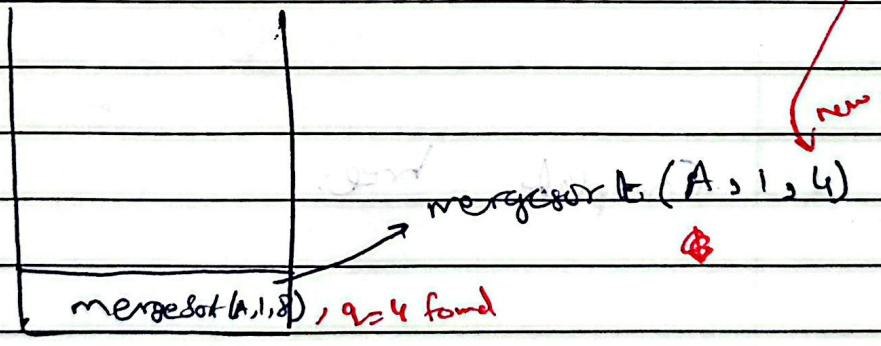
First call inside Mergesort:

p < r, (1 < 8) ✓ yes

so

$q = \lfloor \frac{8+1}{2} \rfloor = 4$

now it calls mergesort again: (lines)



BHATTI

CamScanner

y/date

~~how call call~~

now we invoke mergeSort(A, 1, 4).

r = 4. now

$q = \lfloor \frac{1+4}{2} \rfloor = 2$

mergesort(A, 1, 2) mergesort(A, 3, 4)

& and all this is continued.

Then when we return from e.g. mergesort(A, 1, 4), then the second recursive call (line 6) runs :: mergesort(A, 5, 8)

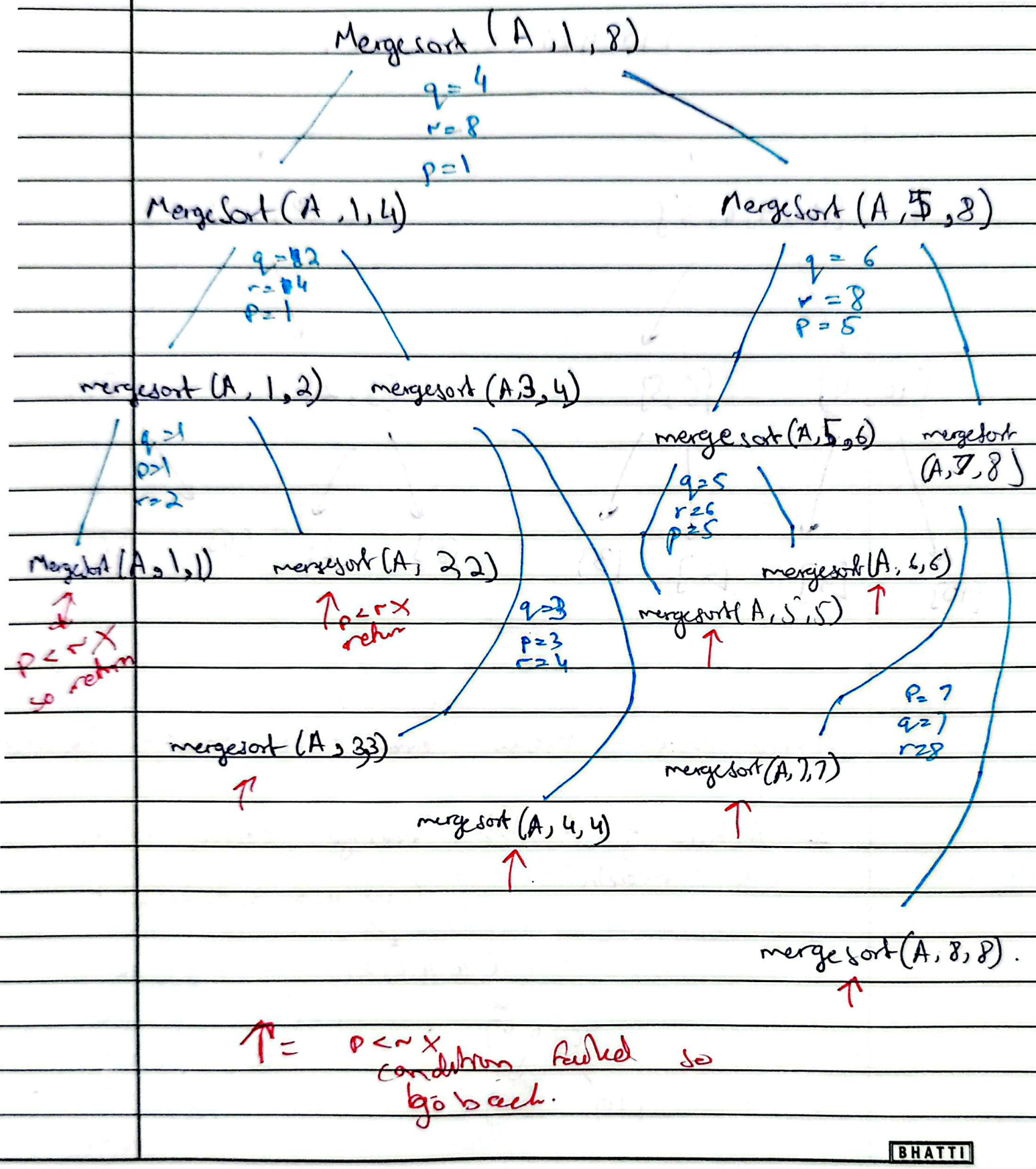
~~q = 6~~,
q = 4.

Complete tree.

BHATTI

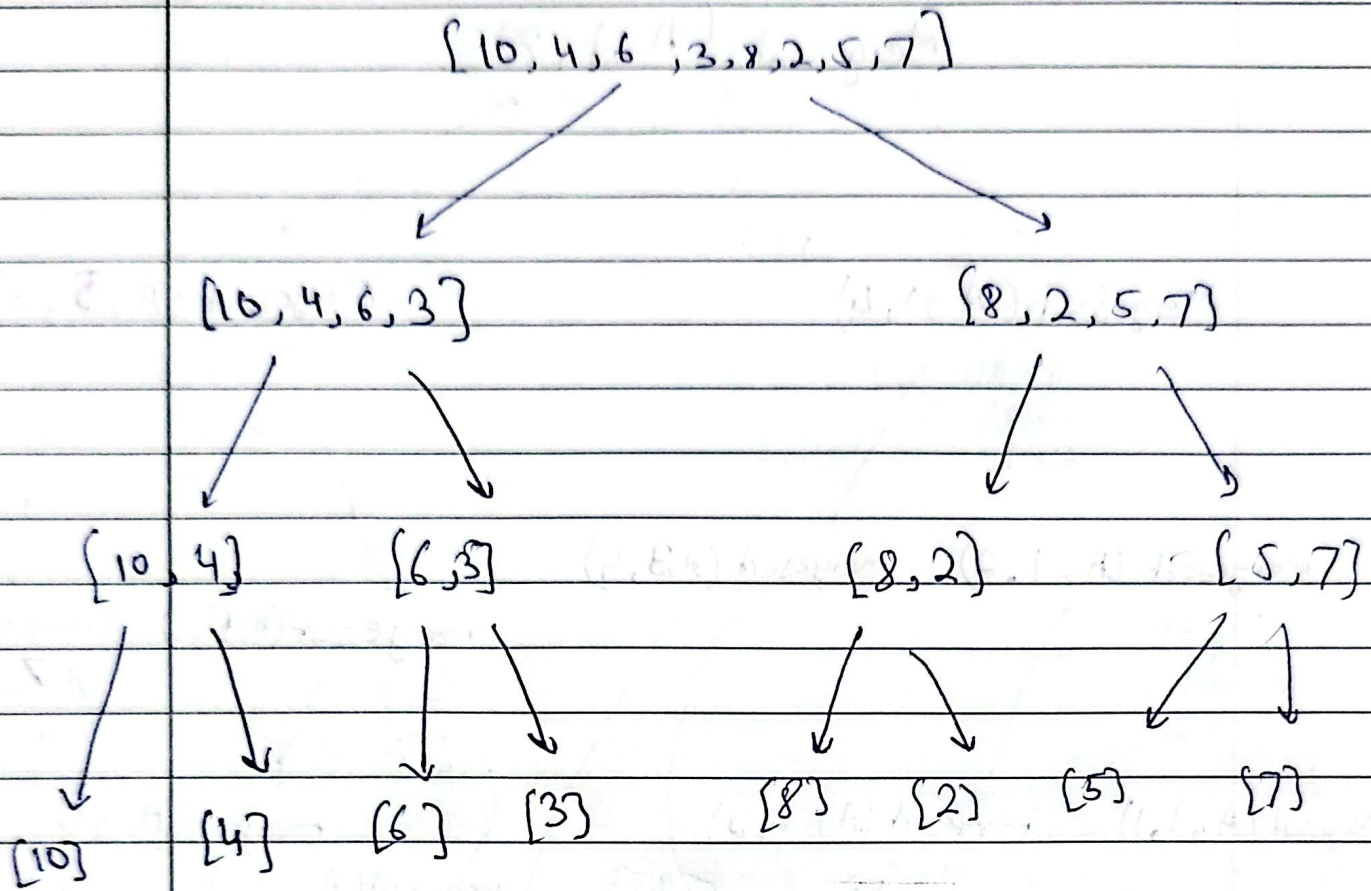
CamScanner

day/date



BHATTI

day/date



After this now comes the merge function.

* Actually the first merge comes when we reach

(10, 4, 6, 3, 8, 2, 5, 7)

(10, 4, 6, 3)

(10, 4)

(10) (4)

This doesn't exist yet. It's just recursive calls.

BHATTI

y/date

now let's see merge for [0] & [4]
(p=1, q=1, r=2)

$n_1 = 1 - 1 + 1 = 1$

$n_2 = 2 - 1 = 1$

for i=1 to n_1:
    L[1] = A[1] = 10
    end loop.

for j=1 to n_2:
    R[1] = A[1] = 4
    end loop.

Here loop only copy the content of both child arrays to L & R.

now comes the comparison loop. (IMP).

for k=p to r
    if L[i] ≤ R[j]
        A[k] = L[i]
        i = i+1
    else A[k] = R[j]
        j = j+1;

what is does is compare the first elements of both arrays & which ever is smaller place it in

BHATTI

/date

kth position in A & only increment that Array's pointer.

e.g.:

i

R = [2, 3, 6]

L = [1, 4, 5].

j

A = [↑].

k is wherever P is, considering (at start currently)

now first R[1] & L[1] compared.

as 1 < 2 so

A = [1 ↑ — —]

k

i

R = [2, 3, 6]

L = [1, 4, 5]

j

BHATTI

te

Again compare:

R[i] & L[j]

as

2 < 4 so:

A[1 2 ...]
k

i 2
R=[2, 3, 6]
L=[1, 4, 5]
0 j

& keep doing this.

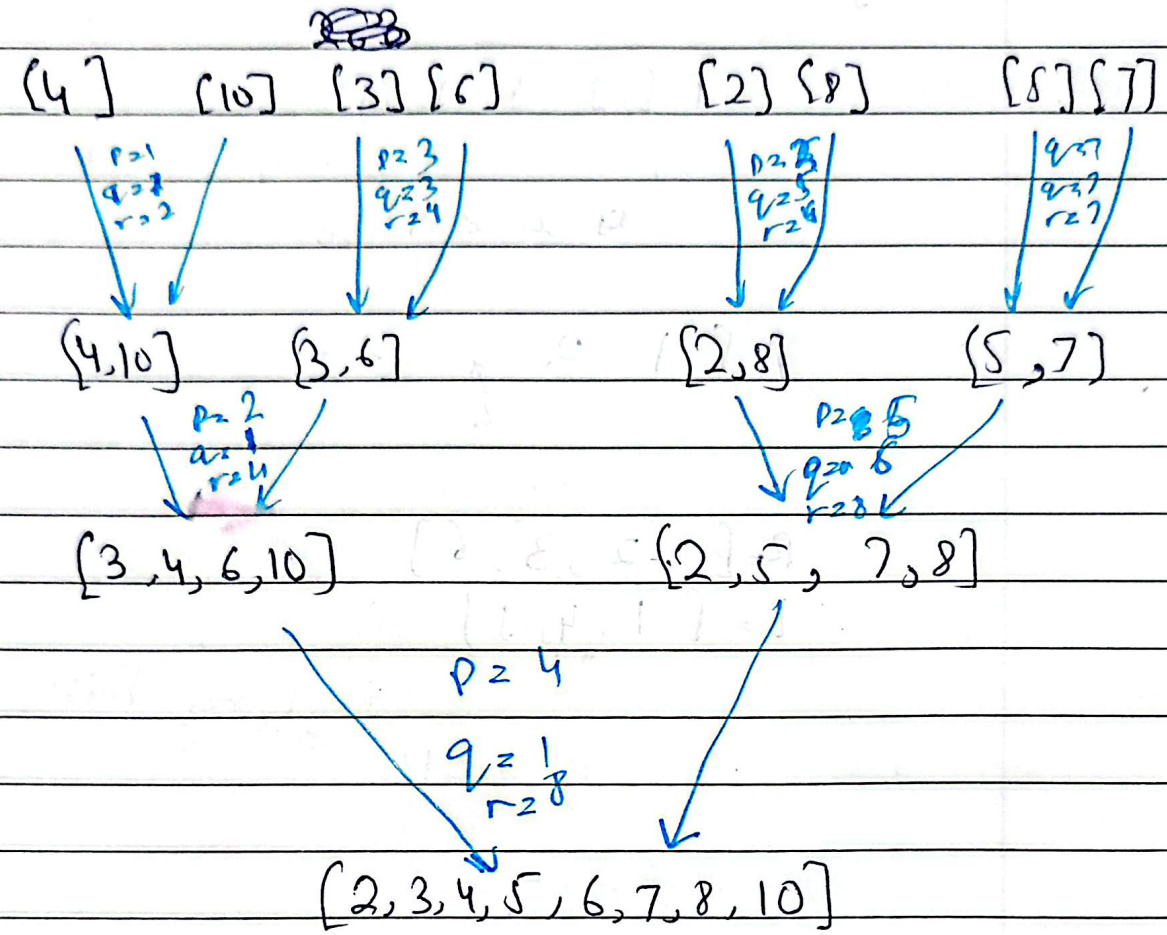
Also remember R & L will always be sorted because they are coming back from childs which run merge itself.

BHATTI

CamScanner

/date

Menging:



BHATTI

y/date

Analysis of mergesort:

If we see the mergesort algo our base case is

If $P >= r$, then return.

so that is $\Theta(1)$ complexity.

we have 2 recursive declarations both running $T(\frac{n}{2})$

& complexity of our merge function = $\Theta(n)$

$$T(n) = \begin{cases} \Theta(1) & \text{if } n=1 \\ 2T(\frac{n}{2}) + \Theta(n) & \text{if } n>1. \end{cases}$$

2 recursive calls of $T(n/2)$.

merge (complexity)

BHATTI

CamScanner

Date

- Now using Recursion tree, Iteration or master method you can find its complexity.

$\neq \Theta(n \log n)$

Time complexity of best, average, worst is same for mergesort, because it divides & merges regardless of the scenario.

- Even in case of a already sorted array like: [1, 2, 5, 7, 12], it will still divide it & then merge it.

so $\Theta(n \log n)$ is for all cases

day/date

A way to make merge sort better:

we just need to add the condition of if our array A is already sorted no need to merge

```
Mergesort (A, p, r) {

If p < r  Then
   q = ⌊ (p+r)/2 ⌋ .

   merge sort (A, p, q)
   merge sort (A, q+1, r)

   if A[q] <= A[q+1] Then
        return
   &

   merge (arr, p, q, r)
}
```

This extra condition shows that array1 should be lower than most of which array2 already sorted, so no need to do step of merge.

BHATTI

CamScanner

In this worst case complexity remain n log n but for best case becomes $\Theta(n)$

$$\text{Q best case} = \begin{cases} \Theta(n) \\ 2T(n/2) + \Theta(1) \end{cases}$$

n/o merge

& and by master theorem case 1, $= \Theta(n)$.

BHATTI

CamScanner